

Explicación técnica del código de machine learning para el precio del oro

Alejandro Ramirez Leidy Paola Patiño

15 de junio de 2026

1. Objetivo del código

El archivo `simulacion_ml_mejorada.py` construye un modelo predictivo para el precio del oro. El código toma datos históricos, crea variables explicativas, compara varios algoritmos de machine learning, selecciona el modelo con menor error en validación temporal y genera pronósticos con simulación Monte Carlo.

2. Carga de datos

El código lee el archivo `historicoauplata.xlsx`. Detecta automáticamente la hoja del oro, la columna de fecha y la columna objetivo. También intenta cargar la plata, precios NI 43-101 y variables macroeconómicas de FRED. Las fechas se ordenan cronológicamente para evitar mezclar pasado y futuro.

3. Variable objetivo

No se predice directamente el precio del oro. Se predice el log-retorno del precio de cierre al siguiente día hábil:

$$r_{t+1} = \ln \left(\frac{P_{t+1}}{P_t} \right). \quad (1)$$

Luego se reconstruye el precio predicho:

$$\hat{P}_{t+1} = P_t e^{\hat{r}_{t+1}}. \quad (2)$$

Esto hace que el modelo trabaje con una variable más estable que el nivel del precio.

4. Construcción de variables

El código crea rezagos del precio del oro:

$$Lag_k(t) = P_{t-k}. \quad (3)$$

También calcula retornos, diferencias, medias móviles y volatilidades:

$$R_k(t) = \frac{P_t}{P_{t-k}} - 1, \quad (4)$$

$$D_k(t) = P_t - P_{t-k}, \quad (5)$$

$$MA_k(t) = \frac{1}{k} \sum_{i=0}^{k-1} P_{t-i}, \quad (6)$$

$$\sigma_k(t) = std(R_1(t-k+1), \dots, R_1(t)). \quad (7)$$

Cuando existe plata, se calcula también la relación oro/plata:

$$Ratio_{Au/Ag}(t) = \frac{P_{oro,t}}{P_{plata,t}}. \quad (8)$$

5. Modelos evaluados

El código compara XGBoost, LightGBM, CatBoost, Random Forest, Extra Trees e Hist-GradientBoosting. Además, usa Optuna para probar automáticamente combinaciones de hiperparámetros y añadir el mejor candidato encontrado. En el trabajo anterior se seleccionó XGBoost porque presentó el menor error promedio; en esta versión se amplió la comparación y el mejor modelo por validación temporal fue LightGBM optimizado por Optuna.

6. Validación temporal

Se usa `TimeSeriesSplit`. Esto significa que cada fold entrena con observaciones antiguas y valida con observaciones posteriores. No se usa partición aleatoria porque en series de tiempo eso produciría fuga de información.

7. Métricas

El código calcula MAE, RMSE, MAPE y R^2 :

$$MAE = \frac{1}{n} \sum |y_i - \hat{y}_i|, \quad (9)$$

$$RMSE = \sqrt{\frac{1}{n} \sum (y_i - \hat{y}_i)^2}, \quad (10)$$

$$MAPE = \frac{100}{n} \sum \left| \frac{y_i - \hat{y}_i}{y_i} \right|. \quad (11)$$

8. Pronóstico recursivo

Para pronosticar a futuro, el modelo parte del último precio observado. Predice el retorno del siguiente día, reconstruye el precio y agrega ese precio a la serie. Después repite el proceso:

$$P_{t+1} = P_t e^{\hat{r}_{t+1}}. \quad (12)$$

Este procedimiento permite extender la trayectoria hasta 2035.

9. Monte Carlo

La simulación Monte Carlo agrega incertidumbre al retorno base:

$$r_t^{MC} = \hat{r}_t + \epsilon_t \sigma_s + J_t. \quad (13)$$

Aquí ϵ_t es un choque aleatorio, σ_s es la volatilidad del escenario y J_t es un salto aleatorio ocasional. Después se calcula el precio simulado:

$$P_t^{MC} = P_{t-1}^{MC} e^{r_t^{MC}}. \quad (14)$$

10. Archivos generados

El código genera archivos CSV, Excel, gráficas PNG y el modelo entrenado en formato joblib. Las salidas principales quedan en `resultados/` y las gráficas en `resultados/graficas/`.

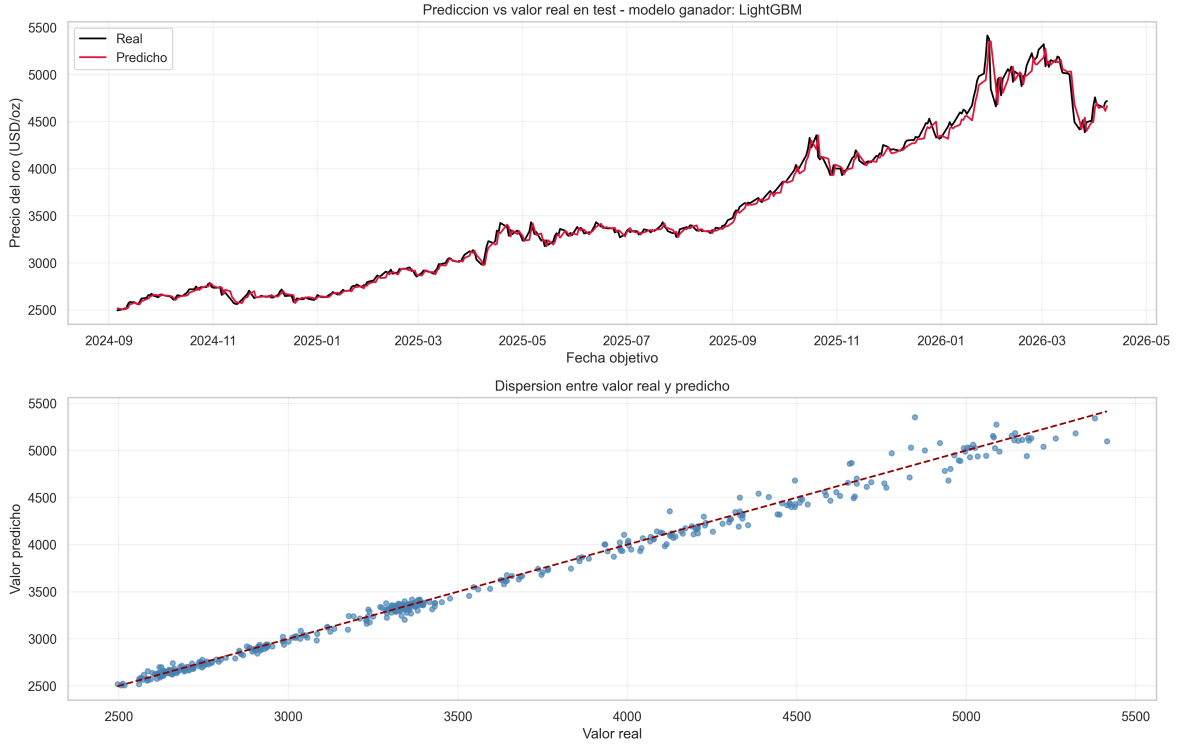


Figura 1: Validación visual del modelo en la ventana de prueba.

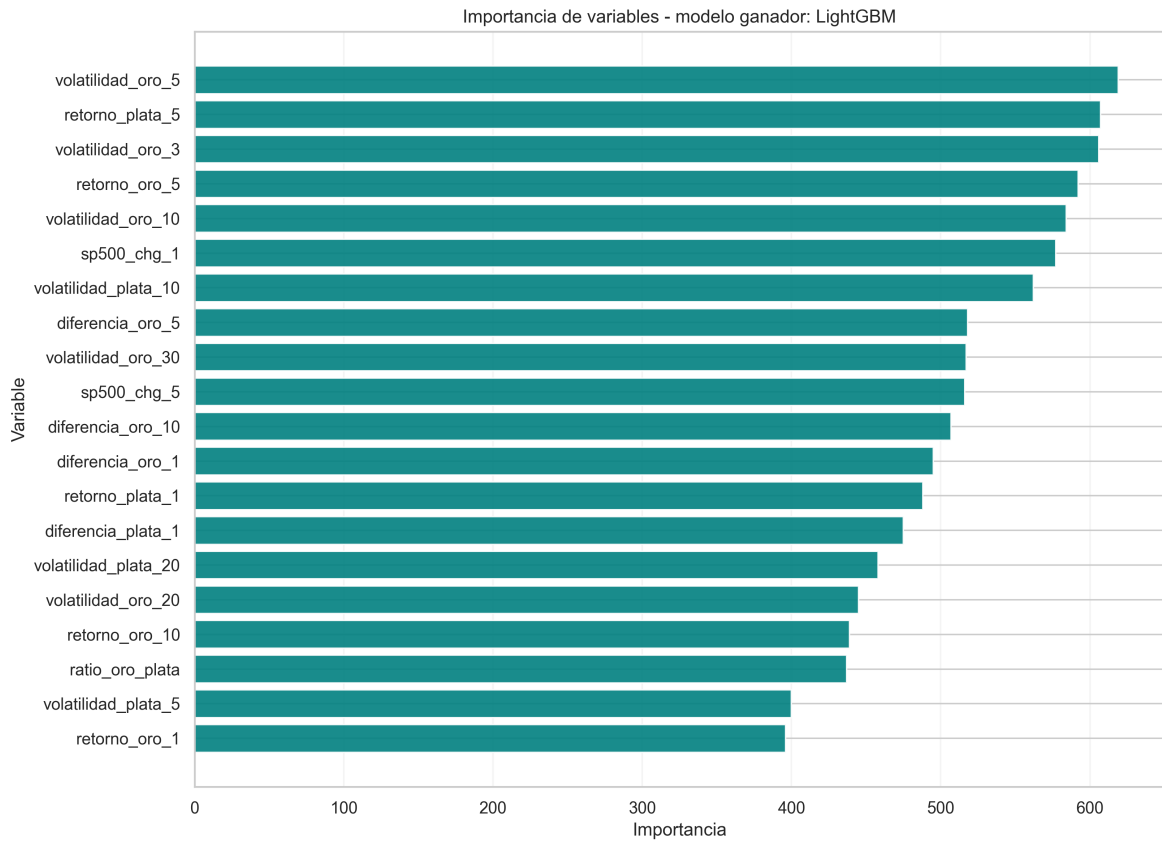


Figura 2: Variables más importantes usadas por el modelo seleccionado.